Структуры данных

Структуры данных

- Массивы
- **Списки**
- Стеки
- Очереди
- **Деревья**
- **Хеш-таблицы**
- …

Односвязный список

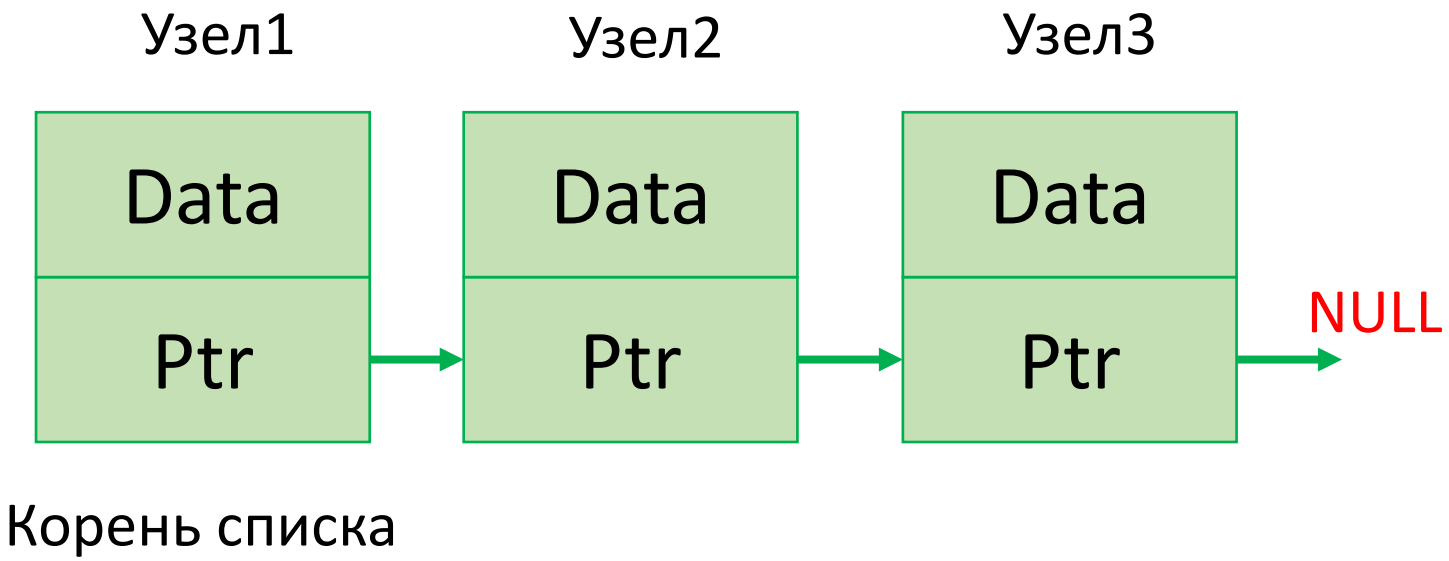
Двусвязных список

Бинарное дерево

АВЛ-дерево

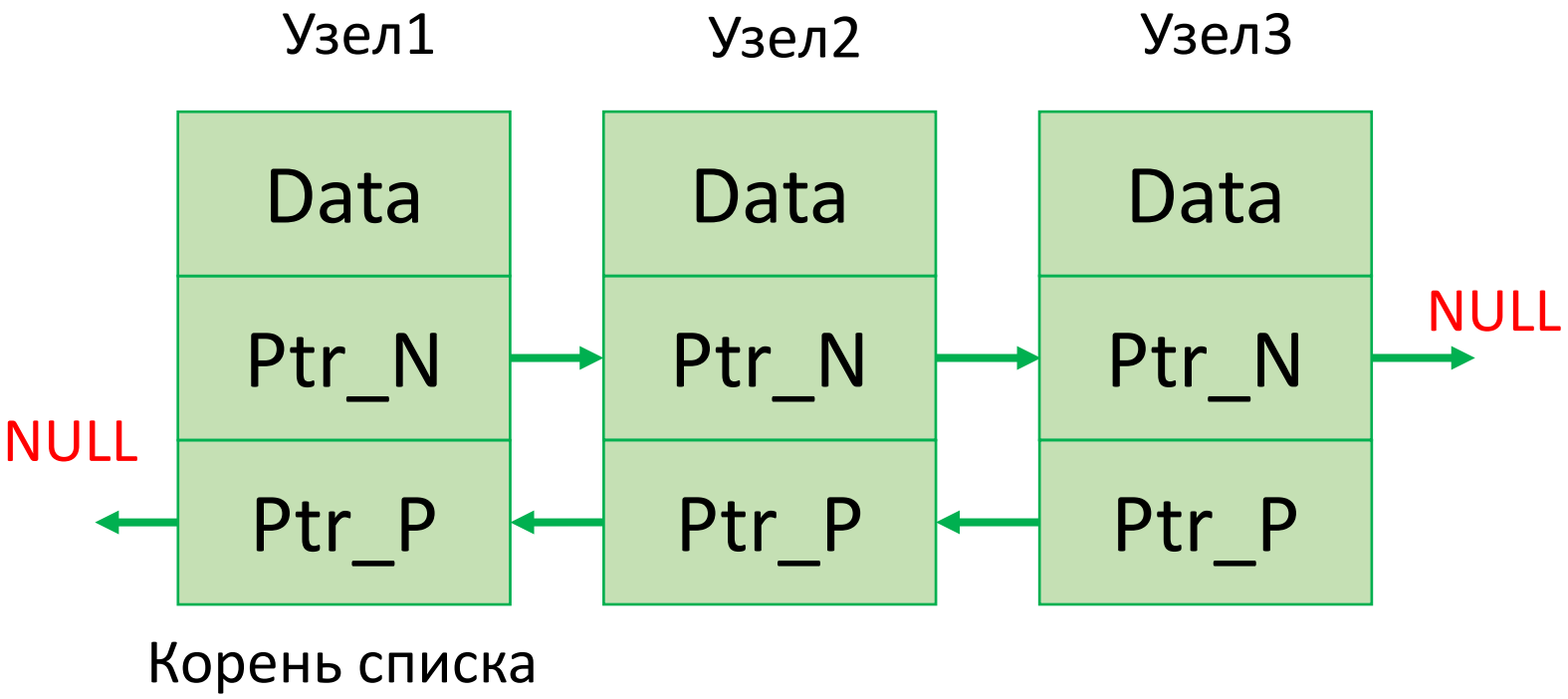
Хеш-таблица

Односвязные списки



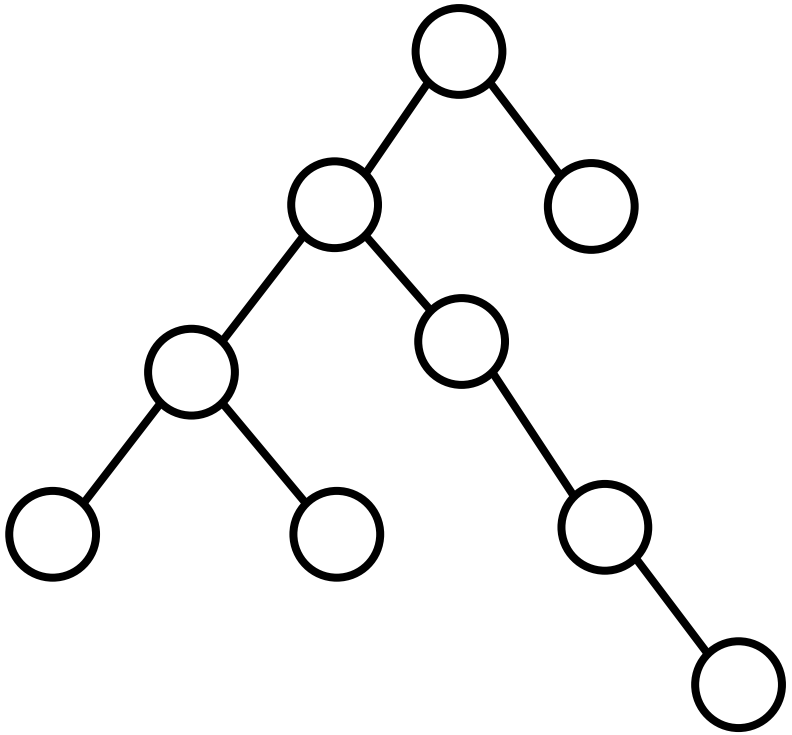
Каждый узел содержит поле указателя на **следующий** узел. Поле указателя последнего узла содержит нулевое значение (указывает на NULL).

Двусвязные списки



Каждый узел содержит два поля указателей: на следующий и на предыдущий узел.
Поле указателя последнего и первого узла содержит нулевое значение (указывает на NULL).

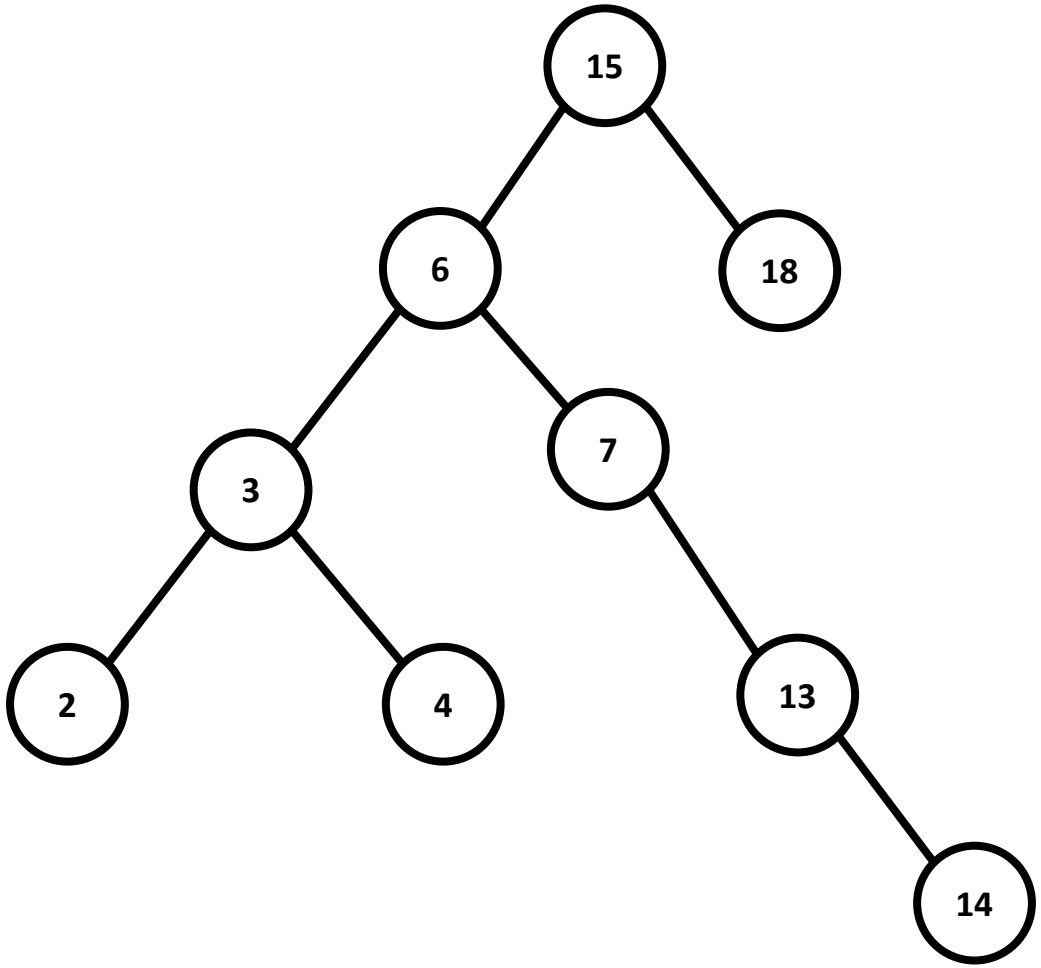
Деревья



Бинарное дерево — это структура данных, в которой каждый узел имеет не более двух потомков
Корень — верхний узел дерева, у которого нет родителя.
Листовой узел (терминальный) — узел, у которого нет потомков (оба указателя NULL)
Высота узла — это длина самого длинного пути от этого узла до листа.

Бинарное дерево поиска: поиск

```
ITERATIVE_TREE_SEARCH(x, k)
while x = NIL и k = key[x]
        do if k < key[x]
                then x ← left[x]
                else x ← right[x]
return x
```



Бинарное дерево поиска: поиск

TREE_SEARCH(x, k)

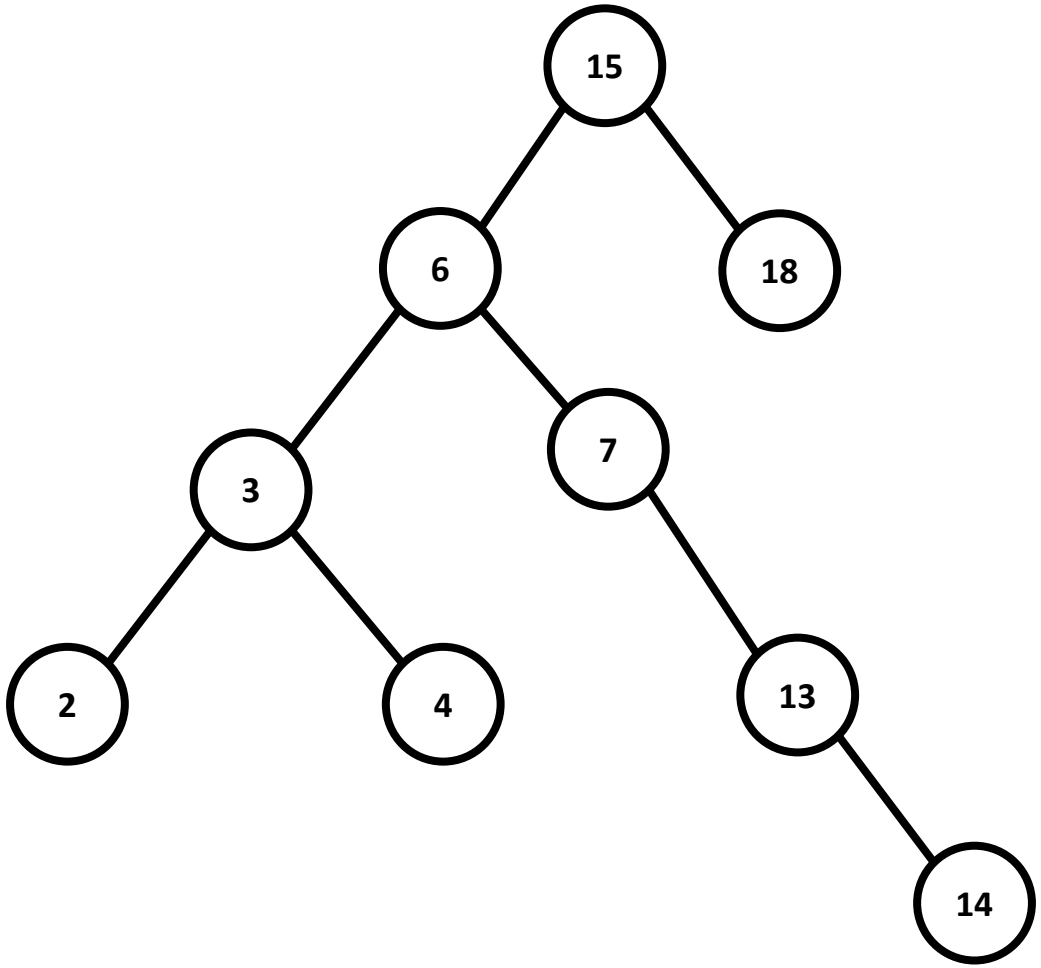
1 **if** x = NIL или $k = key[x]$

2 **then return** x

3 **if** $k < key[x]$

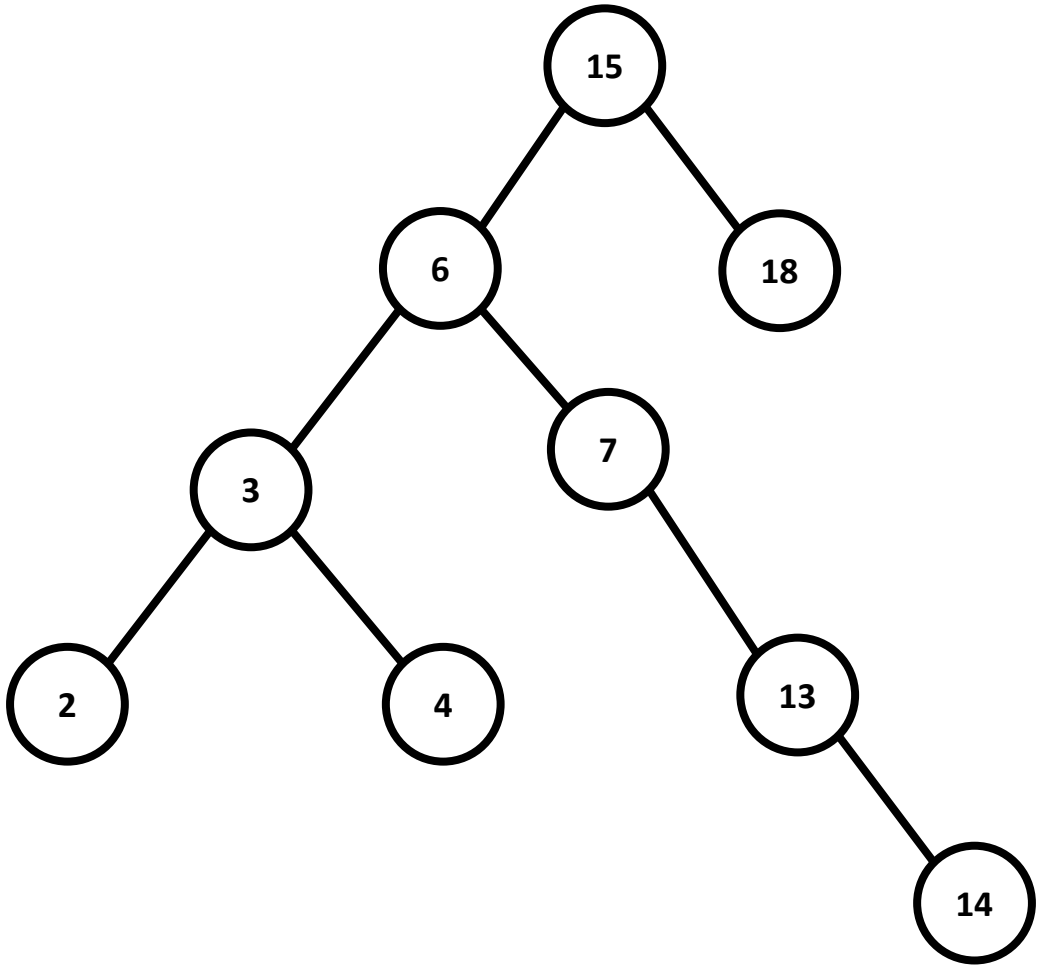
4 **then return** TREE_SEARCH($left[x]$, k)

5 **else return** TREE_SEARCH($right[x]$, k)



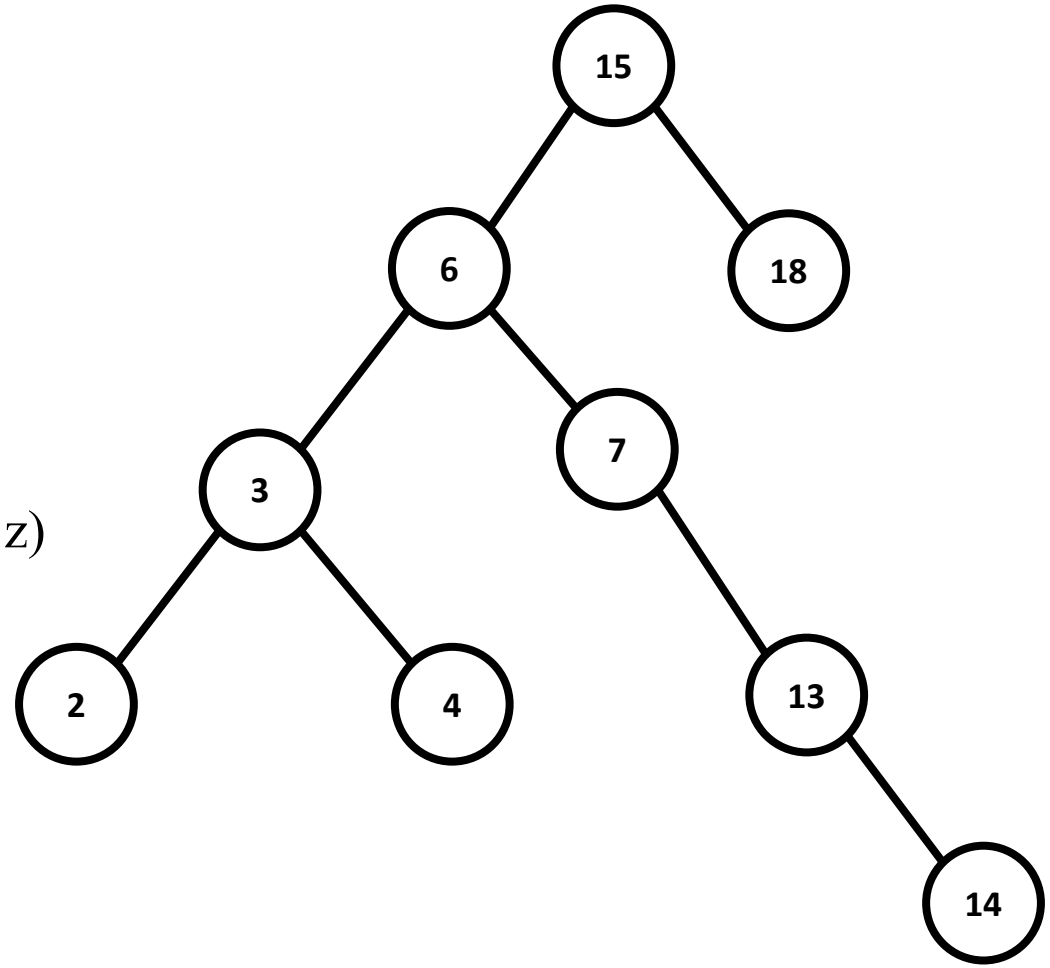
Бинарное дерево поиска: вставка

```
TREE_INSERT(T, z)
y ← NIL
x ← root[T]
while x = NIL
        do y ← x
                if key[z] < key[x]
                        then x ← left[x]
                        else x ← right[x]
p[z] ← y
if y = NIL
        then root[T] ← z  Дерево T — пустое
        else if key[z] < key[y]
                then left[y] ← z
                else right[y] ← z
```

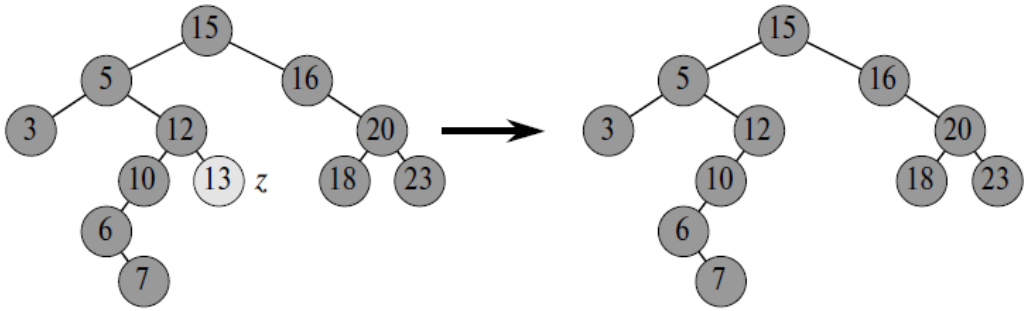


Бинарное дерево поиска: вставка

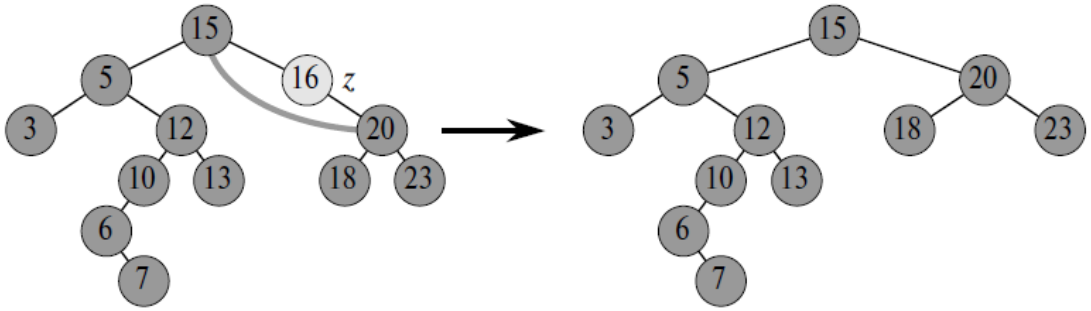
```
TREE_INSERT_RECURSIVE(x, z)
if x = NIL
        then return z
if key[z] < key[x]
        then left[x] ← TREE_INSERT_RECURSIVE(left[x], z)
else
        right[x] ← TREE_INSERT_RECURSIVE(right[x], z)
return x
```



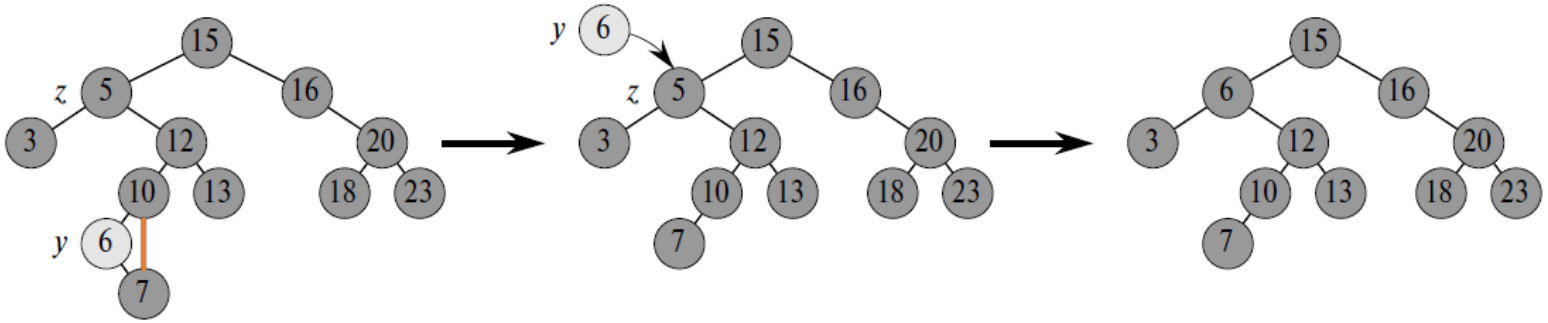
Бинарное дерево поиска: удаление



а)



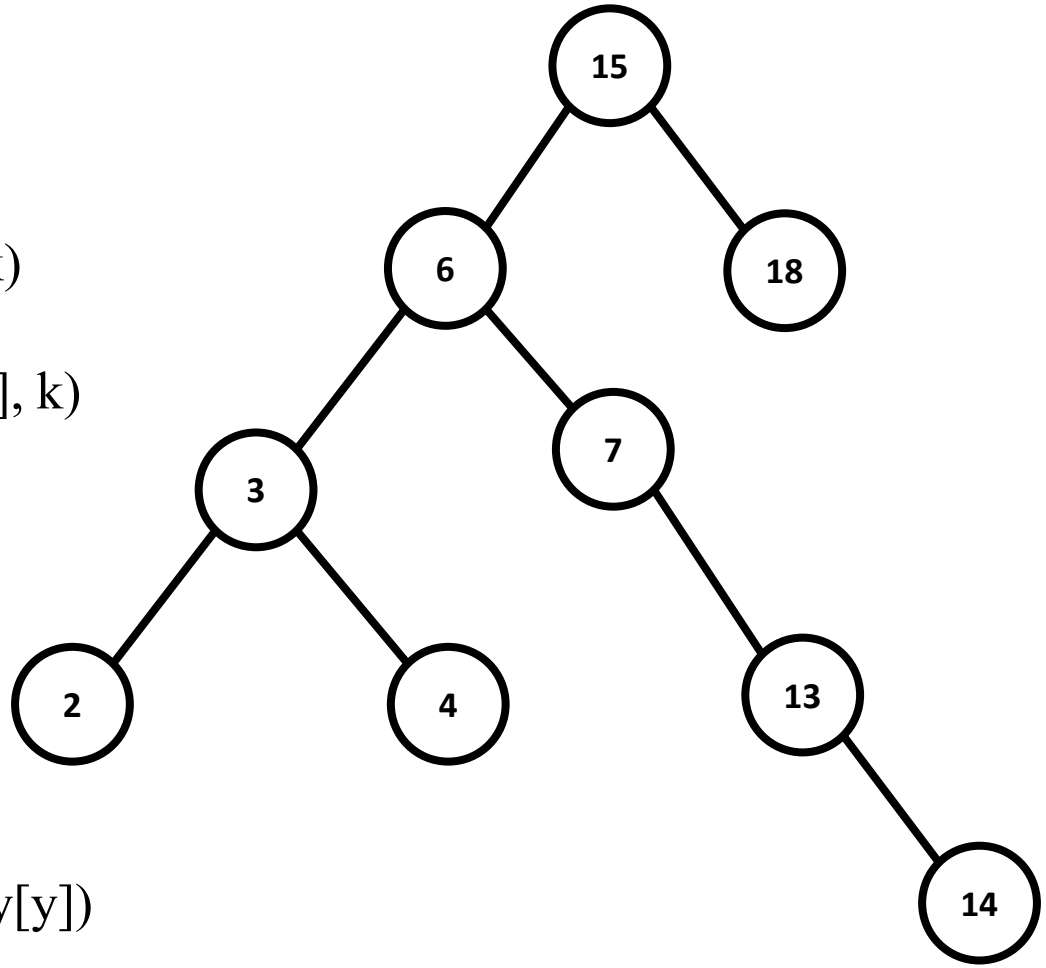
б)



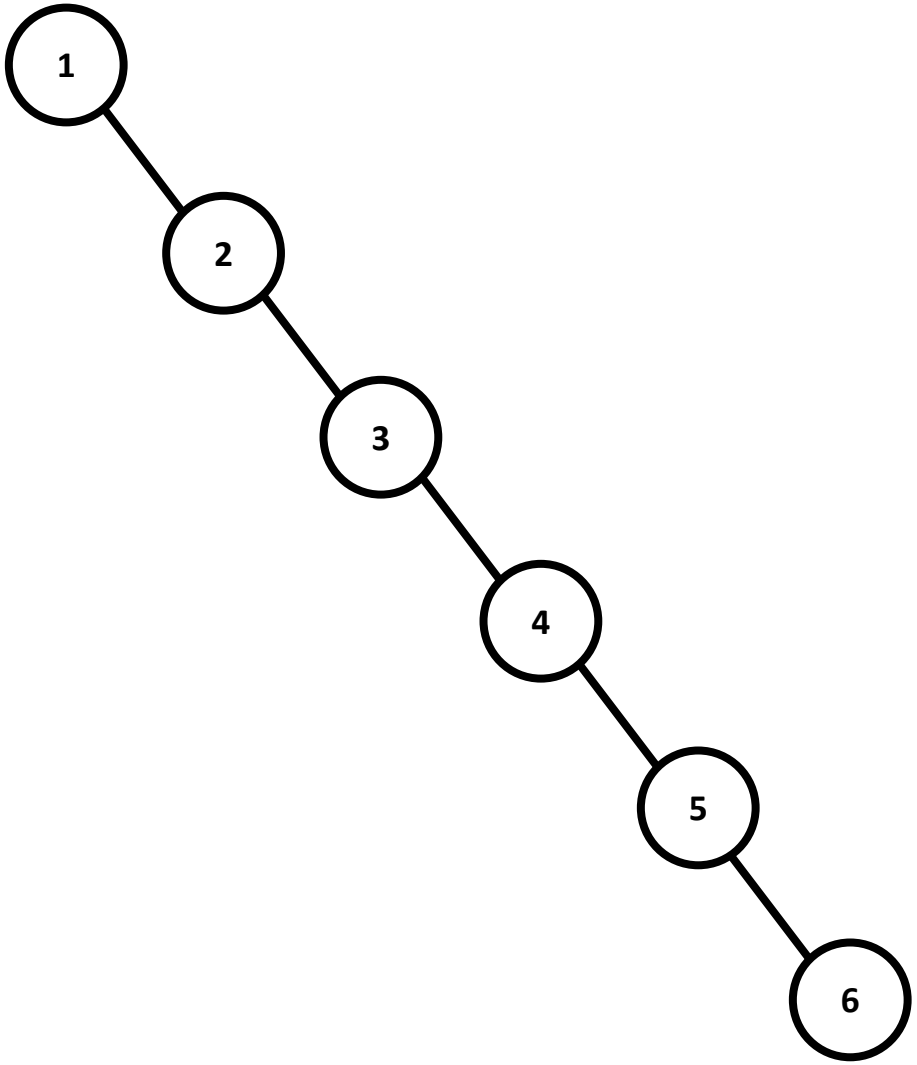
в)

Бинарное дерево поиска: удаление

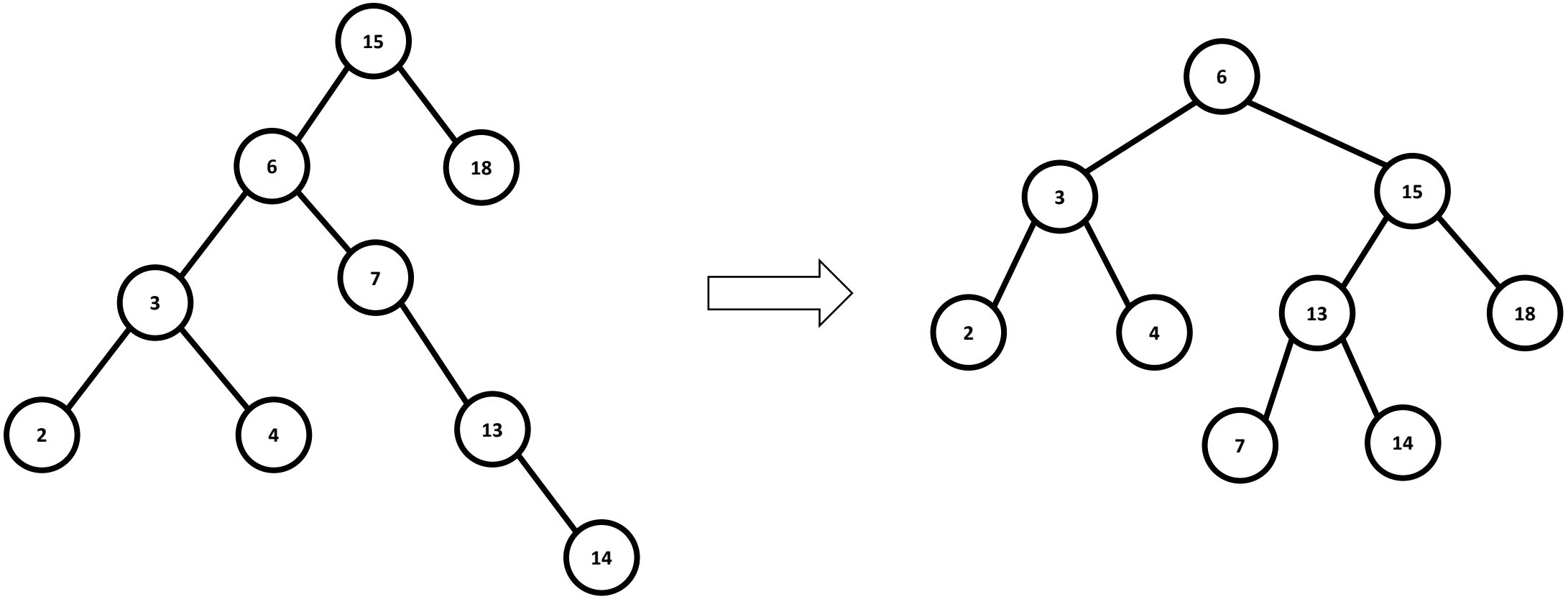
```
TREE_DELETE_RECURSIVE(x, k)
if x = NIL
    then return x
if k < key[x]
    then left[x] ← TREE_DELETE_RECURSIVE(left[x], k)
else if k > key[x]
    then right[x] ← TREE_DELETE_RECURSIVE(right[x], k)
else
    if left[x] = NIL
        then return right[x]
    else if right[x] = NIL
        then return left[x]
    y ← TREE_MINIMUM(right[x])
    key[x] ← key[y]
    right[x] ← TREE_DELETE_RECURSIVE(right[x], key[y])
return x
```



И жили они долго и счастливо (нет)



Сбалансированные деревья



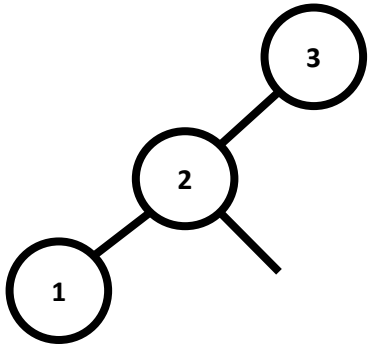
AVL-дерево — это бинарное дерево поиска, в котором для любого узла выполняется условие:

$$| h(left) - h(right) | \leq 1$$

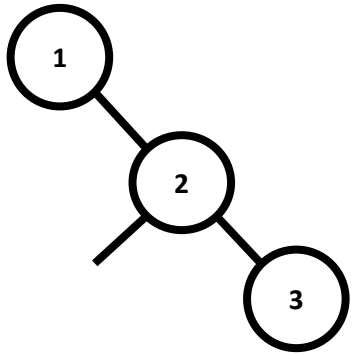
где:

- h(left) — высота левого поддерева
- h(right) — высота правого поддерева

Повороты деревьев

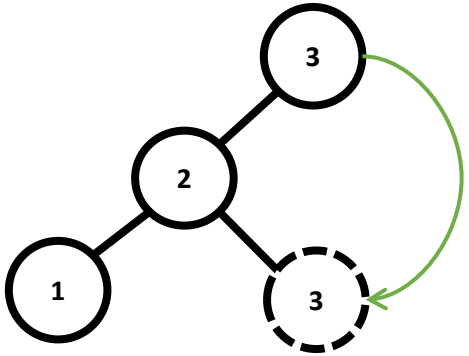


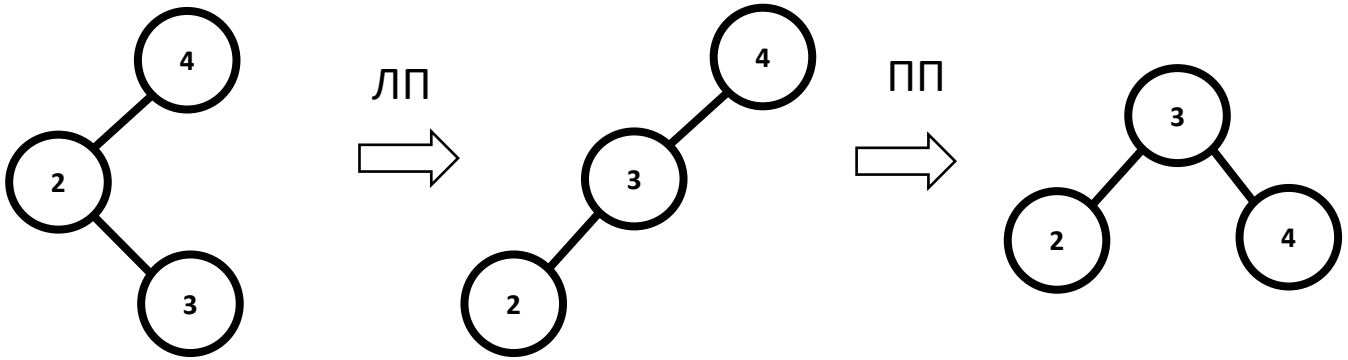
Правый поворот



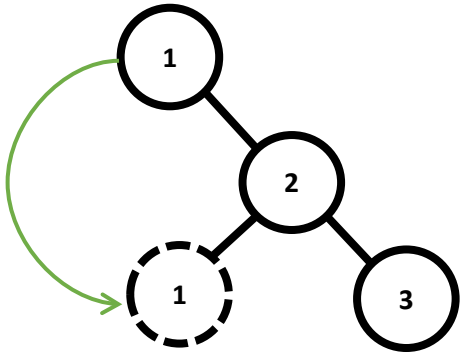
Левый поворот

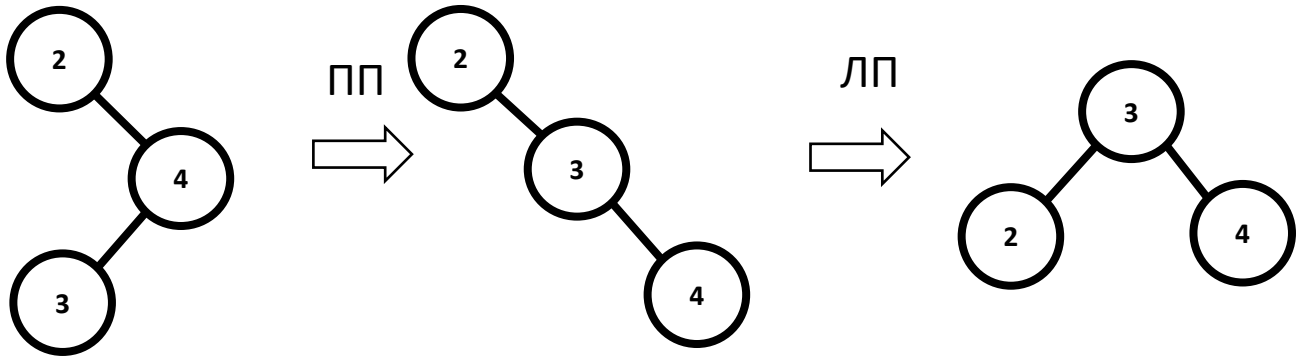
Повороты деревьев





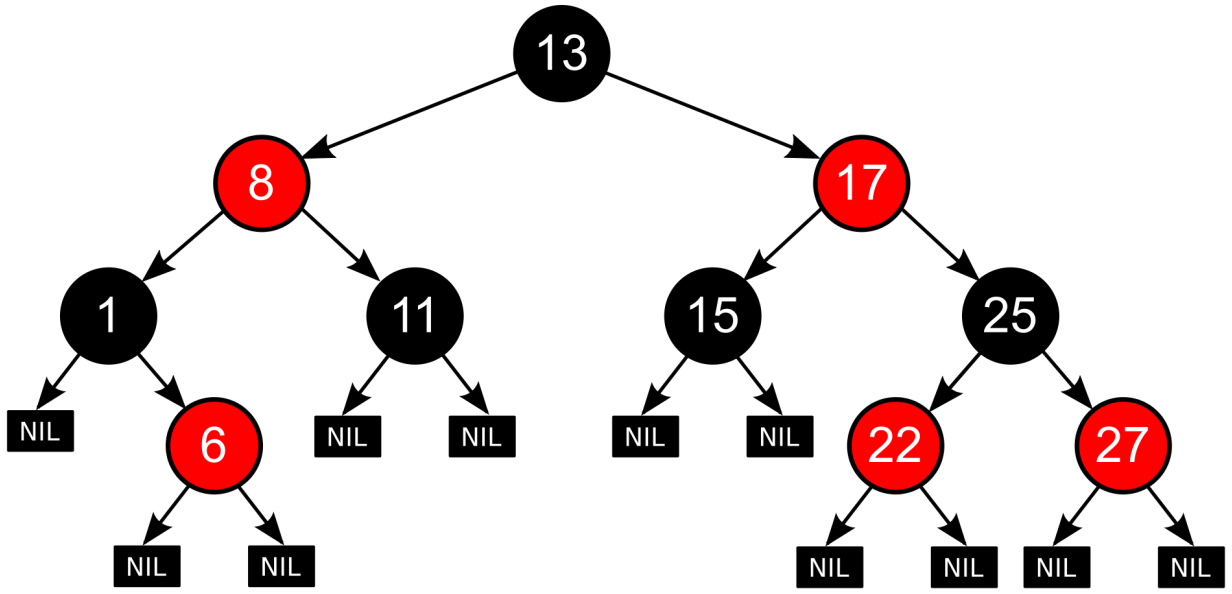
Правый поворот





Левый поворот

Красно-черное дерево: реализация



1. Узел может быть либо красным, либо чёрным и имеет двух потомков;
2. Корень — как правило чёрный. Это правило слабо влияет на работоспособность модели, так как цвет корня всегда можно изменить с красного на чёрный;
3. Все листья, не содержащие данных — чёрные.
4. Оба потомка каждого красного узла — чёрные.
5. Любой простой путь от узла-предка до листового узла-потомка содержит одинаковое число чёрных узлов.

Красно-чёрное дерево — двоичное дерево поиска, в котором каждый узел имеет атрибут *цвета*.

Полный код см. в репозитории